

BAB XII

TRANSACTION CONTROL LANGUAGE (TCL)

12.1 Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa:

1. Memahami konsep *Transaction Control Language* (TCL) sebagai bagian dari SQL yang digunakan untuk mengatur dan mengendalikan proses transaksi pada basis data, termasuk pengelolaan perubahan data.
2. Memahami penggunaan perintah `SAVEPOINT` untuk membuat titik penyimpanan sementara dalam transaksi serta `ROLLBACK` untuk membatalkan perubahan data dan mengembalikan kondisi basis data ke keadaan sebelumnya.
3. Memahami penggunaan perintah `COMMIT` untuk menyimpan seluruh perubahan data secara permanen ke dalam basis data sehingga transaksi menjadi final.

12.2 Dasar Teori

Transaction Control Language (TCL) merupakan bagian dari SQL yang digunakan untuk mengatur dan mengendalikan proses transaksi pada basis data. Transaksi adalah sekumpulan perintah SQL yang dijalankan sebagai satu kesatuan proses. TCL digunakan untuk menjaga konsistensi dan integritas data melalui perintah seperti `COMMIT`, `ROLLBACK`, dan `SAVEPOINT`. Selain itu, TCL juga membantu memastikan data tetap konsisten selama proses transaksi berlangsung (Oracle, 2024).

`SAVEPOINT` adalah perintah dalam TCL yang digunakan untuk membuat titik penyimpanan sementara di dalam suatu transaksi. Titik ini memungkinkan pengguna untuk mengembalikan proses transaksi ke kondisi tertentu tanpa membatalkan seluruh transaksi yang sedang berjalan. Dengan adanya `SAVEPOINT`, pengelolaan transaksi menjadi lebih fleksibel dan terkontrol (Oracle, 2024).

`ROLLBACK` adalah perintah yang digunakan untuk membatalkan perubahan data dalam transaksi yang belum disimpan secara permanen. Perintah ini mengembalikan kondisi *database* ke keadaan sebelum transaksi dijalankan, atau ke

titik tertentu yang ditentukan dengan `SAVEPOINT`. Hal ini membantu menjaga keamanan dan konsistensi data dalam sistem (Oracle, 2024).

`COMMIT` adalah perintah yang digunakan untuk menyimpan seluruh perubahan data secara permanen ke dalam *database*. Setelah `COMMIT` dijalankan, perubahan tidak dapat dibatalkan dan sudah dapat diakses oleh pengguna lain. Selain itu, `COMMIT` juga menghapus `SAVEPOINT` dan mengakhiri transaksi yang sedang berlangsung (Oracle, 2024).

12.3 Hasil dan Pembahasan

Berikut merupakan percobaan *query* berdasarkan modul *section* 18.

1. *Query*

```
UPDATE copy_departments SET manager_id= 101 WHERE  
department_id = 60;
```

Hasil :

```
SQL> UPDATE copy_departments SET manager_id= 101 WHERE department_id = 60;  
1 row updated.
```

Gambar 12.1 Tampilan mengubah `manager_id` pada tabel
`copy_departments`

Penjelasan:

Query tersebut digunakan untuk memperbarui data pada tabel `copy_departments`. Perintah `UPDATE` berfungsi untuk mengubah nilai kolom `manager_id` menjadi “101” pada baris dengan `department_id = 60`. Dengan *query* ini, data dengan ID tersebut akan diperbarui sesuai nilai yang telah ditentukan.

2. *Query*

```
SAVEPOINT one;
```

Hasil :

```
SQL> SAVEPOINT one;  
Savepoint created.
```

Gambar 12.2 Tampilan `SAVEPOINT` setelah proses `UPDATE`

Penjelasan:

Query tersebut digunakan untuk membuat `SAVEPOINT` dengan nama “one” setelah proses `UPDATE` dijalankan. `SAVEPOINT` berfungsi sebagai titik penyimpanan sementara dalam transaksi yang memungkinkan perubahan data dikembalikan ke kondisi tertentu menggunakan perintah `ROLLBACK`. Dengan adanya `SAVEPOINT` ini, perubahan sebelumnya masih dapat dibatalkan tanpa mempengaruhi seluruh transaksi.

3. **Query:**

```
INSERT INTO copy_departments(department_id,  
department_name, manager_id, location_id)  
VALUES(130, 'Estate Management', 102, 1500);
```

Hasil:

```
SQL> INSERT INTO copy_departments(department_id, department_name, manager_id, location_id) VALUES(130, 'Estate Management', 102, 1500  
);  
1 row created.
```

Gambar 12.3 Tampilan memasukan data ke tabel `copy_departments`

Penjelasan:

Query tersebut digunakan untuk menambahkan data baru ke dalam tabel `copy_departments` menggunakan perintah `INSERT INTO`. Dengan *query* ini, data baru akan tersimpan ke dalam tabel `copy_departments`.

4. **Query:**

```
UPDATE copy_departments  
SET department_id = 140;
```

Hasil:

```
SQL> UPDATE copy_departments  
2 SET department_id = 140;  
  
10 rows updated.
```

Gambar 12.4 Mengubah seluruh nilai `department_id` pada tabel `copy_departments` menggunakan `UPDATE`

Penjelasan:

Query tersebut digunakan untuk mengubah nilai `department_id` pada tabel `copy_departments` menjadi “140” menggunakan perintah `UPDATE`. Karena *query* tidak menggunakan klausa `WHERE`, maka seluruh data pada kolom `department_id` di tabel akan berubah menjadi ”140”.

5. *Query*:

```
ROLLBACK TO SAVEPOINT one;
```

Hasil:

```
SQL> SAVEPOINT one;
Savepoint created.

SQL> UPDATE copy_departments
  2 SET department_id = 140;

10 rows updated.

SQL> ROLLBACK TO SAVEPOINT one;
Rollback complete.
```

Gambar 11.5 Pengembalian transaksi ke SAVEPOINT one menggunakan ROLLBACK

Penjelasan:

Query tersebut digunakan untuk mengembalikan transaksi ke SAVEPOINT one yang telah dibuat sebelumnya. Dengan perintah ROLLBACK TO SAVEPOINT, perubahan data setelah SAVEPOINT dibuat akan dibatalkan, sedangkan perubahan sebelum SAVEPOINT tetap tersimpan.

6. *Query:*

```
COMMIT;
```

Hasil:

```
Savepoint created.  
  
SQL> UPDATE copy_departments  
2 SET department_id = 140;  
  
10 rows updated.  
  
SQL> ROLLBACK TO SAVEPOINT one;  
  
Rollback complete.  
  
SQL> COMMIT;  
  
Commit complete.
```

Gambar 12.6 Menyimpan transaksi secara permanen
menggunakan COMMIT

Penjelasan:

Query tersebut digunakan untuk menyimpan seluruh perubahan transaksi secara permanen ke dalam database menggunakan perintah COMMIT. Setelah COMMIT dijalankan, seluruh perubahan data yang sebelumnya dilakukan tidak dapat dibatalkan lagi dengan ROLLBACK. Perintah ini biasanya digunakan setelah proses INSERT, UPDATE, atau DELETE selesai dan data sudah dianggap benar serta siap disimpan secara final.

12.4 Latihan

1. Menggunakan perintah TCL untuk mengelola transaksi data dalam tabel.

Query:

```
INSERT INTO mahasiswa (nim, nama, jurusan) VALUES
(2407112931, 'William James Moriarty',
'Informatika');
SAVEPOINT william;
DELETE FROM mahasiswa
WHERE nama = 'William James Moriarty';
ROLLBACK TO SAVEPOINT William;
COMMIT;
```

Hasil:

```
SQL> INSERT INTO mahasiswa (nim, nama, jurusan) VALUES (2407112931, 'William James Moriarty', 'Informatika');
1 row created.
SQL> SAVEPOINT william;
Savepoint created.
SQL> DELETE FROM mahasiswa WHERE nama = 'William James Moriarty';
0 rows deleted.
SQL> DELETE FROM mahasiswa WHERE nim = 2407112931;
1 row deleted.
SQL> INSERT INTO mahasiswa (nim, nama, jurusan) VALUES (2407112931, 'William James Moriarty', 'Informatika');
1 row created.
SQL> SAVEPOINT william;
Savepoint created.
SQL> DELETE FROM mahasiswa WHERE nama = 'William James Moriarty';
1 row deleted.
SQL> ROLLBACK TO SAVEPOINT william;
Rollback complete.
SQL> COMMIT;
Commit complete.
```

Gambar 12.7 Penggunaan perintah TCL untuk mengelola transaksi pada tabel mahasiswa

Penjelasan:

Query tersebut digunakan untuk mengelola transaksi pada tabel mahasiswa. Dimulai dengan menambahkan data mahasiswa “William James Moriarty”, kemudian dibuat `SAVEPOINT` sebagai titik penyimpanan sementara. Setelah itu dilakukan `DELETE` pada data tersebut, namun dibatalkan menggunakan `ROLLBACK TO SAVEPOINT` sehingga data mahasiswa kembali seperti semula. Terakhir, perintah `COMMIT` digunakan untuk menyimpan seluruh perubahan secara permanen.

12.5 Kesimpulan

Berdasarkan praktikum, dapat disimpulkan bahwa *Transaction Control Language* (TCL) merupakan bagian dari SQL yang digunakan untuk mengatur dan mengendalikan proses transaksi pada basis data. TCL berperan penting dalam menjaga konsistensi dan integritas data melalui pengelolaan transaksi secara terstruktur, sehingga setiap perubahan data dapat diproses dengan lebih aman dan terkontrol.

Selain itu, perintah `SAVEPOINT` digunakan untuk membuat titik penyimpanan sementara di dalam transaksi. Dengan adanya `SAVEPOINT`, transaksi dapat dikembalikan ke kondisi tertentu tanpa harus membatalkan seluruh proses yang sedang berjalan, sehingga pengelolaan transaksi menjadi lebih fleksibel dan efisien.

Selanjutnya, `ROLLBACK` berfungsi untuk membatalkan perubahan data yang belum disimpan secara permanen dan mengembalikan *database* ke kondisi sebelumnya. `ROLLBACK` juga dapat digunakan bersama `SAVEPOINT` untuk kembali ke titik tertentu sesuai kebutuhan. Sementara itu, `COMMIT` digunakan untuk menyimpan seluruh perubahan data secara permanen ke dalam *database*, sehingga perubahan dapat dilihat oleh pengguna lain dan tidak dapat dibatalkan lagi menggunakan `ROLLBACK`.